

SEO Automation Module

Integration Guide for ticketgol.com

Version 3.1 – 2026-05-08 (Emergent-free)

SEO Automation Module — Integration Guide for ticketgol.com

*Portable SEO module v3.0 — extracted from the GoLevents codebase. Stack: **FastAPI (Python 3.11+)** + **MongoDB** + **React 18**. This module runs as a **separate microservice** alongside your existing Laravel/MySQL stack.*

1. Prerequisites

Your server (or a separate VPS / Render / Railway instance) needs:

- **Python 3.11+**
- **MongoDB 6+** (free tier on MongoDB Atlas works fine — 512 MB)
- **Node 18+ + yarn** (only if you want the admin UI, not strictly required)
- A reachable domain or sub-domain for the FastAPI service, e.g. `https://seo-api.ticketgol.com`

You will also need API keys for:

Provider	Required for	Where to get it
Anthropic API	Claude (master copywriter IT + FAQ generator)	https://console.anthropic.com/settings/keys
Google AI Studio	Gemini Pro (JSON-LD enrichment + Vision) + Nano Banana (hero image)	https://aistudio.google.com/apikey (free tier available)
Perplexity API	FAQ + sameAs + GeoCoordinates + Team Verifier	https://www.perplexity.ai/settings/api
DeepL API (Free or Pro)	IT → EN/ES translation	https://www.deepl.com/pro-api
DataForSEO	Keyword research	https://app.dataforseo.com
Resend (optional)	Email alerts on cost overruns	https://resend.com/api-keys

Nota: il modulo è **completamente indipendente da Emergent**. Tutte le call vanno direttamente alle API ufficiali dei provider. Una **sola** key Google AI Studio copre sia Gemini Pro (text/JSON-LD/vision) sia Nano Banana (image generation), perché entrambi sono accessibili tramite lo stesso endpoint `generativelanguage.googleapis.com`.

2. Get the code

Clone the repo and run the extraction script. It copies only the 47 files of the SEO module into your project tree:

```
git clone <REPO_URL> golevents-source
cd /path/to/your/ticketgol/project

bash ../golevents-source/copy_seo_module.sh .
rm -rf ../golevents-source
```

You should now see, inside your project:

```
backend/
├─ routes/          # 5 files: seo_admin, seo_targets, seo_tools,
seo_intelligence, cost_observatory
├─ services/        # 22 files: 17 seo_* + 5 api_* (cost observatory)
frontend/src/
├─ pages/admin/seo/  # 12 pages (6 SEO core + 6 Intelligence Hub)
├─ components/       # 6 components (4 public + 2 admin)
├─ utils/seoHero.js  # 1 helper
SEO_PORTABLE_MODULE.md # full reference docs
```

3. Install dependencies

Backend (Python)

```
cd backend
pip install fastapi uvicorn motor pydantic httpx apscheduler rapidfuzz cryptogra
phy python-dotenv pillow
pip freeze > requirements.txt
```

No emergent integrations, no EMERGENT_LLM_KEY. The SEO module talks directly to Anthropic / Google AI Studio / Perplexity / DeepL / DataForSEO HTTP endpoints.

Frontend (only if you build the admin UI)

```
cd frontend
yarn add lucide-react sonner react-router-dom recharts
```

4. Environment variables

Create `backend/.env` :

```
MONGO_URL=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net
DB_NAME=ticketgol_seo
CORS_ORIGINS=https://ticketgol.com,https://www.ticketgol.com

# Encryption key for storing 3rd-party API keys at rest
SEO_FERNET_KEY=<generate with: python -c "from cryptography.fernet import
Fernet; print(Fernet.generate_key().decode())">

# Public base URL where the FastAPI service is reachable
BASE_URL=https://seo-api.ticketgol.com

# Provider API keys – Option A: set them here as env vars (priority over DB-
stored keys)
# Option B (recommended): leave empty here and configure them via /admin/seo/
api-tools UI
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
PERPLEXITY_API_KEY=
DEEPL_API_KEY=
DATAFORSEO_LOGIN=
DATAFORSEO_PASSWORD=
RESEND_API_KEY=
```

Do not commit the `.env` file. Add it to `.gitignore`.

Create `frontend/.env` (only if you ship the admin UI):

```
REACT_APP_BACKEND_URL=https://seo-api.ticketgol.com
```

5. Register routers in your FastAPI server

In your `backend/server.py`:

```
from fastapi import FastAPI
from routes import seo_admin, seo_targets, seo_tools, seo_intelligence, cost_observatory

app = FastAPI()
app.include_router(seo_admin.router)
app.include_router(seo_targets.router)
app.include_router(seo_tools.router)
app.include_router(seo_intelligence.router)
app.include_router(cost_observatory.router)
```

The module also expects an `admin_auth.verify_admin_token` dependency. Provide a minimal implementation:

```
# backend/routes/admin_auth.py
from fastapi import Header, HTTPException
import os

ADMIN_API_KEY = os.environ["ADMIN_API_KEY"] # set in .env

async def verify_admin_token(authorization: str = Header(...)):
    token = authorization.replace("Bearer ", "").strip()
    if token != ADMIN_API_KEY:
        raise HTTPException(status_code=401, detail="Invalid token")
    return True
```

Add to `.env`:

```
ADMIN_API_KEY=<choose a long random string, e.g. openssl rand -hex 32>
```

6. Required entity fields

The module reads from these MongoDB collections in the `DB_NAME` database. Your Laravel app stays the source of truth in MySQL — you will sync the relevant entities here:

```
events:    { slug, home_team, away_team, league, league_slug, stadium, city,
country, sort_date }
teams:     { slug, name, league, stadium, city, country, logo_url }
leagues:   { slug, name, country }
```

You can either:

- **Option A (recommended)**: write a small cron in Laravel that pushes events/teams/leagues from MySQL to this Mongo every 30 min via a `POST /api/sync/upsert` endpoint you implement;
- **Option B**: have the FastAPI service read directly from your MySQL via `pymysql` (replace the MongoDB calls with SQL queries — non-trivial, only do this if you really want a single DB).

The simplest path is **Option A** with Mongo as a thin cache layer.

7. Run

```
cd backend
uvicorn server:app --host 0.0.0.0 --port 8001
```

Behind nginx / Caddy, point `https://seo-api.ticketgol.com` to port 8001.

For production:

```
uvicorn server:app --host 0.0.0.0 --port 8001 --workers 4
```

(or use `gunicorn` with uvicorn workers, or systemd, or supervisor).

8. Configure 3rd-party API keys

After the service is running, hit:

```
curl -X POST https://seo-api.ticketgol.com/api/seo/admin/tools/anthropic/key \
-H "Authorization: Bearer <ADMIN_API_KEY>" \
-H "Content-Type: application/json" \
-d '{"api_key":"<your-anthropic-key>"}
```

Repeat for: `gemini`, `perplexity`, `deepl`, `dataforseo`, `resend`.

Or, if you ship the admin UI, open `https://seo-api.ticketgol.com/admin/seo/api-tools` in the browser.

9. Public API endpoints (call these from Laravel)

All endpoints require `Authorization: Bearer <ADMIN_API_KEY>` unless flagged `PUBLIC`.

Generate SEO content for an entity

```
POST /api/seo/targets/event/<event_id>/generate
GET  /api/seo/jobs/<job_id>                    # poll until status == "done"
GET  /api/seo/targets/event/<event_id>        # read generated meta
POST /api/seo/targets/event/<event_id>/publish
```

Response sample (after publish):

```
{
  "seo_meta": {
    "title": "...",
    "description": "...",
    "h1": "...",
    "intro_html": "...",
    "main_html": "...",
    "faq": [...],
    "json_ld": {...},
    "hreflang": {...},
    "hero_image_url": "https://seo-api.ticketgol.com/api/seo/uploads/<file>.png"
  }
}
```

Bulk generate

```
POST /api/seo/targets/bulk-generate-league # body: { "league_slug": "la-liga",
"limit": 50 }
```

Public read endpoints (NO auth — safe to call from front-end)

```
GET /api/seo/intelligence/faq/<type>/<slug>/public
GET /api/seo/intelligence/trust-score/<type>/<slug>
GET /api/seo/uploads/<filename>
```

Cost Observatory

```
GET /api/seo/cost-observatory/overview
GET /api/seo/cost-observatory/providers
GET /api/seo/cost-observatory/logs
GET /api/seo/cost-observatory/balance
POST /api/seo/cost-observatory/alerts/run-checks
```

Full endpoint list: see `SEO_PORTABLE_MODULE.md` section "API Endpoints".

10. Calling from Laravel

Example Laravel service class:


```
// app/Services/SeoEngine.php
namespace App\Services;

use Illuminate\Support\Facades\Http;

class SeoEngine
{
    private string $base;
    private string $token;

    public function __construct()
    {
        $this->base = config('services.seo_engine.url');
        $this->token = config('services.seo_engine.token');
    }

    public function generateForEvent(int $eventId): string
    {
        $r = Http::withToken($this->token)
            ->post("{ $this->base }/api/seo/targets/event/{ $eventId }/generate");
        return $r->json('job_id');
    }

    public function getJob(string $jobId): array
    {
        return Http::withToken($this->token)
            ->get("{ $this->base }/api/seo/jobs/{ $jobId }")
            ->json();
    }

    public function getMeta(int $eventId): array
    {
        return Http::withToken($this->token)
            ->get("{ $this->base }/api/seo/targets/event/{ $eventId }")
            ->json('seo_meta');
    }
}

```

config/services.php :

```
'seo_engine' => [
    'url' => env('SEO_ENGINE_URL'),
    'token' => env('SEO_ENGINE_TOKEN'),
],

```

.env of Laravel:

```
SEO_ENGINE_URL=https://seo-api.ticketgol.com
SEO_ENGINE_TOKEN=<same ADMIN_API_KEY as above>
```

In your Blade template:

```
<head>
  <title>{{ $event->seo_meta['title'] ?? $event->fallback_title }}</title>
  <meta name="description" content="{{ $event->seo_meta['description'] ??
  '' }}">
  @if (!empty($event->seo_meta['json_ld']))
    <script type="application/ld+json">{!! json_encode($event-
  >seo_meta['json_ld']) !!}</script>
  @endif
</head>
```

11. Schedulers (optional)

The module ships an APScheduler that you can wire into your FastAPI startup to run alert checks every 30 minutes:

```
from contextlib import asynccontextmanager
from services.api_alerts import run_all_alert_checks
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.cron import CronTrigger

scheduler = AsyncIOScheduler()

@asynccontextmanager
async def lifespan(app):
    scheduler.add_job(run_all_alert_checks, CronTrigger(minute="*/30"))
    scheduler.start()
    yield
    scheduler.shutdown()

app = FastAPI(lifespan=lifespan)
```

12. Smoke test

```
TOKEN=$ADMIN_API_KEY
curl -s https://seo-api.ticketgol.com/api/seo/admin/tools \
  -H "Authorization: Bearer $TOKEN" | jq

curl -s https://seo-api.ticketgol.com/api/seo/cost-observatory/overview \
  -H "Authorization: Bearer $TOKEN" | jq
```

If both return JSON, the service is up and ready to be used from Laravel.

13. File inventory

47 files total, distributed as:

- **5** routes (`backend/routes/`)
- **22** services (`backend/services/`)
- **12** admin pages (`frontend/src/pages/admin/seo/` , optional)
- **6** components (`frontend/src/components/` , public + admin)
- **1** util (`frontend/src/utills/seoHero.js`)
- **1** docs (`SEO_PORTABLE_MODULE.md`)

For the full file list and per-file purpose, read `SEO_PORTABLE_MODULE.md` .

14. Support / troubleshooting

Symptom	Likely cause	Fix
401 Unauthorized on every call	Wrong <code>ADMIN_API_KEY</code>	Re-check <code>.env</code> and the <code>Authorization</code> header
<code>cryptography.fernet.InvalidToken</code>	<code>SEO_FERNET_KEY</code> changed	Use the same Fernet key everywhere; never rotate without re-encrypting
Pipeline stuck on <code>step: claude</code>	Missing <code>ANTHROPIC_API_KEY</code>	Set it in <code>.env</code> OR paste it in <code>/admin/seo/api-tools</code> (slug <code>anthropic</code>)
Pipeline stuck on <code>step: gemini</code>	Missing <code>GEMINI_API_KEY</code>	Set it in <code>.env</code> OR paste it in <code>/admin/seo/api-tools</code> (slug <code>gemini</code>) — same key for Nano Banana
Hero image not generated	Missing/expired Google AI Studio key	Get a free one at https://aistudio.google.com/apikey
Cost overview always 0	No data yet	Run a Bulk Generate first; the dashboard fills as jobs run

Module version: 3.0 (2026-05-08) **Python:** 3.11+ **Node:** 18+ **License:** proprietary — internal use only